1. **Consider the following code snippet and identify the basic operation first. Write the time complexity (Big O) of the code. Show your calculations.**

```
FOR i FROM 0 TO n-1
FOR j FROM 0 TO n-1
FOR k FROM 0 TO n-1
PRINT i, j, k
```

```
FOR i FROM 1 TO n
PRINT i

FOR j FROM 1 TO n²
PRINT j
```

```
sum = 0
FOR i FROM 0 TO n-1
FOR j FROM 0 TO n-1
sum = sum + 1
```

```
i = n
WHILE i > 1
PRINT i
i = i / 2
```

2. **Consider the following unsorted array of employee IDs:**

   [105, 210, 145, 320, 180, 250, 195, 410, 300, 125]

   a) Use Linear Search to find employee ID 300.

   b) Show each comparison performed.

   c) State the number of comparisons required.

   d) Explain the best-case and worst-case time complexity of Linear Search.

3. **Given the following sorted array:**

   [5, 10, 15, 20, 25, 30, 35, 40, 45, 50]

   a) Apply Binary Search to find the value 35.

   b) Show each iteration and midpoint calculation.

   c) State the number of comparisons required.

   d) Explain the time complexity of Binary Search.

4. **A social media platform wants to count the total number of likes received by each post.**

   Pseudo-code:

   ```
   FOR each post IN posts
   totalLikes = totalLikes + post.likes
   ```

   a) Identify the basic operation.

   b) How many times is the basic operation executed for n posts?

   c) Determine the time complexity.

   d) Explain how the execution time changes when the number of posts doubles.

5. **A lecturer wants to compare every student's assignment submission with every other student's submission to detect possible plagiarism.**

Pseudo-code:

```
FOR i FROM 0 TO n-1
FOR j FROM 0 TO n-1
Compare(student[i], student[j])
```

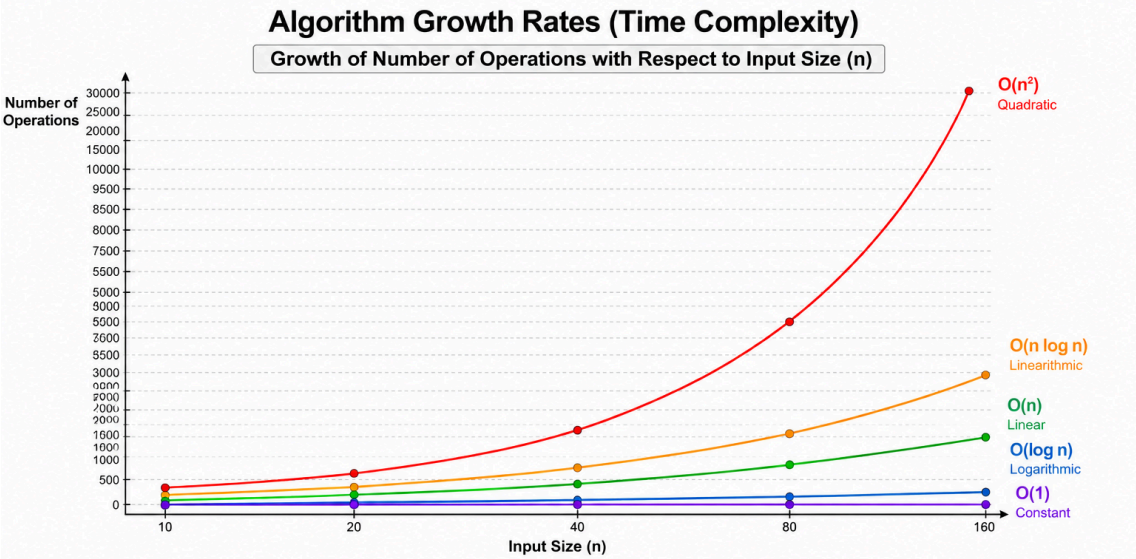
a) Identify the basic operation.

b) Determine the number of comparisons performed.

c) Find the time complexity.

d) Explain why this approach becomes expensive for large classes.

6. **Study the graph below, which illustrates the relationship between problem size (n) and the number of operations performed by different algorithms.**



Input Size (n)	10	20	40	80	160
O(1)	1	1	1	1	1
O(log n)	3	4	5	6	7
O(n)	10	20	40	80	160
O(n log n)	33	86	213	506	1172
O(n²)	100	400	1600	6400	25600

Note: The values are illustrative (not exact mathematical values) and are used to show relative growth.

A software company is developing a student information system. The system currently manages 1,000 student records, but the number of records is expected to increase significantly over the next few years.

The development team is considering the following algorithms:

Algorithm	**Time Complexity**
Algorithm A	$O(n^2)$
Algorithm B	O(n log n)
Algorithm C	O(log n)

Using the graph and your understanding of time complexity, answer the following questions.

A. Rank the three algorithms from most efficient to least efficient when the number of records becomes very large.
B. Based on the graph, explain how the growth rate of Algorithm A differs from Algorithm B as the input size increases.
C. If the number of student records increases from 1,000 to 100,000, which algorithm would be most suitable? Justify your answer using the graph.
D. Explain why an algorithm that performs well for small datasets may become unsuitable for large datasets.
E. The company plans to expand the system to support more than 1 million records in the future. Which complexity classes shown in the graph would be most suitable for such large-scale systems? Explain your reasoning.